

Programowanie Obiektowe

Laboratorium 1

Java – podstawy języka i narzędzia programistyczne

Sprawozdanie w formacie PDF oraz pliki z kodem proszę wysłać przez platformę Teams lub na adres mailowy podany przez prowadzącego zajęcia (wyłącznie w przypadku problemów z oprogramowaniem Teams). Sprawozdanie powinno zawierać **imię i nazwisko autora/autorki, numer albumu i datę**. Również **nazwy przesyłanych plików** (sprawozdanie, archiwa ZIP z kodem itp.) muszą zawierać **nazwisko i imię autora/autorki**. Tekst wyróżniony w ten sposób oznacza rzeczy, które muszą znaleźć się w sprawozdaniu – przede wszystkim zrzuty ekranu i wnioski.

Cel ćwiczenia

Celem ćwiczenia jest zapoznanie studentów z:

- podstawami składni języka Java, w szczególności mechanizmem hermetyzacji i implementacją konstruktorów kopiujących,
- narzędziami używanymi do kompilacji i uruchamiania programów.

Zadanie 1. Pierwszy program w języku Java – kompilacja i uruchomienie (2,5 pkt)

1. Proszę utworzyć klasę **Pixel**, reprezentującą piksel na ekranie. Klasę tę wyposażymy w następujące składowe:
 - Prywatne pole **luminance** typu **int**, odpowiadające jasności piksela.
 - Publiczne stałe (**final**) pole **location** typu **java.awt.Point**, odpowiadające położeniu piksela na ekranie (patrz **Wskazówka** poniżej).
 - Publiczny konstruktor przyjmujący trzy liczby całkowite: jasność piksela, jego współrzędną poziomą i pionową.
 - Publiczny konstruktor kopiujący, tworzący *w pełni niezależną* instancję klasy **Pixel** na podstawie innego obiektu tej klasy.
 - Publiczną metodę **String toString()**, zwracającą w estetycznej formie informacje o jasności i położeniu piksela.

Wskazówka. Każdy plik, w którym korzystamy z obiektów bibliotecznej klasy **Point**, musi zawierać przed definicją naszych klas wiersz: **import java.awt.Point;**

2. Utworzyć publiczną klasę **Main**, zawierającą jedynie metodę `public static void main(String[] args)`. Można to zrobić w osobnym pliku lub w tym samym, w którym zapisana jest klasa **Pixel**, o ile nie została ona zadeklarowana jako publiczna. W metodzie **main** należy kolejno:
 - Utworzyć obiekt klasy **Pixel**.
 - Wyświetlić stan tego obiektu, korzystając z metody `System.out.println`.Aby przetestować kod, należy go najpierw skompilować, wydając polecenie:
`javac Nazwa_pliku_z_metoda_main.java`
Jakie pliki z rozszerzeniem `.class` pojawiły się w katalogu?
Uruchomić program, wydając polecenie:
`java Nazwa_klasy_z_metoda_main`
(Przy nazwie klasy nie podajemy rozszerzenia `class`.) Fragment zrzutu konsoli z wynikiem działania programu zamieścić w sprawozdaniu.
3. Do metody **main** dodać wiersz powodujący odczytanie (np. w celu wyświetlenia) wartości pola **luminance** utworzonego piksela. Spróbować skompilować program. Zamieścić w sprawozdaniu komunikat wygenerowany przez kompilator i skomentować przyczyny niepowodzenia. „Niepoprawny” wiersz należy następnie **wykomentować (aby nie przeszkadzał w dalszej pracy nad projektem), ale go nie usuwać**.
4. W klasie **Pixel** utworzyć publiczne metody dostępne:
 - `getLuminance` – zwracającą jasność piksela,
 - `setLuminance` – modyfikującą jasność.

Zadanie 2. Zabezpieczanie pól obiektów przed modyfikacją (1 pkt)

W metodzie **main** należy kolejno:

1. W funkcji **main** stworzyć nowy obiekt klasy **Point** (aby to się udało, w pliku zawierającym kod tej funkcji należy importować `java.awt.Point`). Następnie przypisać ten obiekt do pola **location** naszego piksela. Spróbować skompilować program. Zamieścić w sprawozdaniu komunikat wygenerowany przez kompilator. Wykomentować wprowadzone polecenie uniemożliwiające kompilację.
2. Utworzyć kopię pierwszego piksela, wykorzystując w tym celu konstruktor kopiujący. Zmienić jasność nowego piksela, używając metody `setLuminance`. Wyświetlić informacje o obu pikselach.
3. Przypisać nową wartość do jednej ze współrzędnych drugiego piksela (np. `location.x`) i ponownie wyświetlić informację o obu pikselach. Porównać ten przypadek z opisanym w pierwszym punkcie:
 - Dlaczego przypisanie jako lokalizacji nowego obiektu klasy **Point** nie było możliwe, a zmiana wartości jednego z pól tego obiektu – tak?
 - Czy taka implementacja klasy **Pixel** jest bezpieczna? Jeśli nie, to co należałoby w niej zmienić? (Proponowanej zmiany nie trzeba implementować, wystarczy ją krótko opisać.)

- Czy konstruktor kopiujący działa prawidłowo, tj. czy oba piksele są niezależne? Innymi słowy, czy zmiana położenia jednego z nich nie powoduje zmiany położenia drugiego?

Ostateczny plik z kodem należy dołączyć do sprawozdania. Jeśli projekt składa się z dwóch plików, przed dołączeniem należy je spakować do archiwum ZIP.

Zadanie 3. *Visual Studio Code* – edycja, kompilacja, uruchamianie i debugowanie kodu języka Java (1 pkt)

Uruchamiamy środowisko *Visual Studio Code* – z menu systemu lub wpisując w terminalu **code&**. Otwieramy stworzony przez nas plik.

Nasz program kompilujemy i uruchamiamy. Wyniki pojawią się w panelu *TERMINAL* w dolnej części okna. Następnie tworzymy pułapkę w wierszu, w którym po raz pierwszy informacja o pikselu wypisywana jest na ekran. Uruchamiamy program w trybie debugowania. Bieżący wiersz jest wyróżniony strzałką wyświetlaną na lewo od numeru wiersza. Na zrzucie ekranu panelu *VARIABLES* proszę zaprezentować wartości wszystkich pól naszej klasy, łącznie ze strukturą obiektu **location**.

Należy pozwolić wejść debuggerowi do wnętrza metody **getLuminance** (klawisz *F11*), a następnie pozwolić jej wykonać się do końca (klawisz *F10*). Jaka nowa informacja pojawiła się w panelu *VARIABLES*?